Tiny Recursive Reasoning Models for Industrial Rostering: An Inductive, Verifier-Gated Proposer with Exact-Solver Fallback

NIRAJ PANKHANIA
COFORGE
NIRAJ.PANKHANIA@COFORGE.COM

Abstract

Tiny Recursive Models (TRM) and their predecessors (HRM, PTRM) solve hard puzzles with sub-million-parameter networks by recursing a small core over latent "reasoning" and "answer" states. To date they have been demonstrated almost exclusively on puzzle benchmarks (Sudoku, Maze, ARC) and are widely criticised as *transductive* and narrow, limiting industrial uptake. We present, to our knowledge, the first measured benchmark of a tiny recursive reasoning model on an industrial workforce-scheduling task, framed as constraint-based shift rostering. Our system differs from canonical TRM/PTRM in three ways: it is *inductive* (no per-instance identity embedding, evaluated on unseen instances); it replaces the learned Q-head with a *deterministic verifier* for inference-time best-of-N selection; and it wraps the proposer in an *exact-solver fallback* (OR-Tools CP-SAT), making the deployed system sound by construction. On commodity CPU we reproduce the central TRM claim—recursion with deep supervision is the performance driver at fixed parameter count (+23 points on rostering)—and show verifier-gated scaling reaching 91.5% feasibility while the Q-head plateaus near 82%. The verifier-gated hybrid attains 100% feasibility while avoiding the exact solver on 89% of instances and running $5\times$ faster end-to-end. A multi-seed pilot hardens these results and reports an honest negative: under a fixed cheap training budget the realised economic speedup falls with problem size, even though a $\sim$0.2M-parameter recursive proposer decisively beats an off-the-shelf 0.5B small language model (0% feasible, $\sim$30,000$\times$ slower per attempt). We conclude the pattern is *sound and safe* but that its economic case at scale remains to be proven.

1 Introduction

Recent "tiny reasoning" models challenge the assumption that competitive reasoning requires large models. The Hierarchical Reasoning Model (HRM) [2], the Tiny Recursive Model (TRM) [1], and the Probabilistic TRM (PTRM) [3] achieve strong results on Sudoku, Maze and ARC with networks under 10M—sometimes under 1M—parameters by *recursing* a single small core over two latent states: an evolving answer y and a latent reasoning state z, trained with deep supervision and truncated backpropagation through time.

Two features have kept these models in the research lab. First, the public demonstrations are *transductive*: each training instance carries a learned identity embedding, so the model can only answer instances it has seen, which is incompatible with most production workflows. Second, the only reported tasks are abstract puzzles, leaving open whether the mechanism transfers to a constrained industrial problem with a real cost model.

This paper addresses both. We take the recommended deployment pattern from the recursive-reasoning literature—a tiny proposer behind a cheap verifier with an exact-solver fallback—and instantiate it, *inductively*, on workforce shift rostering: a canonical constraint-satisfaction problem

[0]Preprint. Under review.

across business process services, airline crew operations, and financial back-office scheduling. Our contributions are empirical and applied-systems, not a new architecture:

1. The first measured benchmark, to our knowledge, of a tiny recursive reasoning model on an industrial scheduling task (Section 5).
2. An *inductive* (identity-free) variant that generalises to unseen instances, directly addressing the transductivity critique (Section 3).
3. *Verifier-gated* best-of-N selection in place of the learned Q-head, with the measured gap that justifies it (Section 5.2).
4. A quantified proposer-plus-exact-solver hybrid—feasibility, solver-call reduction, latency—plus a multi-seed pilot that reports both the robustness wins and an honest economic negative at scale (Section 5.4).

2 Background and Related Work

Recursive reasoning models. HRM [2] couples two recurrent modules at different timescales. TRM [1] simplifies this to a single small network recursed over (y, z) with deep supervision and an EMA of the weights, outperforming much larger models on ARC-AGI. PTRM [3] injects Gaussian noise per recursion step and runs K rollouts, selecting among them with a learned Q-head. All three are demonstrated on puzzles and use per-instance identity embeddings, making them transductive.

Neural rostering. Learning-augmented scheduling is an established field that we position against but do not supersede. Prior work uses a deep network to guide branch-and-bound for personnel rostering [4], and a DNN to repair nurse schedules towards feasibility [5]. Neither is a recursive reasoning model, inductive-by-design, or equipped with verifier-gated test-time scaling and an exact-solver guarantee.

Our positioning. Relative to both lines, we contribute a *tiny recursive* model, trained *inductively*, with *verifier-gated* inference-time scaling and an *exact-solver fallback* that makes the system sound by construction, evaluated on feasibility, solver-call reduction and latency. The underlying primitives (recursion, PTRM-style noise, verifier best-of-N, sound abstention) are from the cited literature; the novelty is their combination, the inductive setup, and the industrial measurement.

3 Method

Model. We use a single 2-layer MLP-mixer core [6] (the TRM-MLP variant: token-mixing plus channel-mixing SwiGLU), recursed over two latent states y (answer logits) and z (latent reasoning). One recursion *block* performs n latent updates $z \leftarrow \mathrm{core}(x_{\mathrm{emb}}, y, z)$ followed by one answer update $y \leftarrow \mathrm{core}(x_{\mathrm{emb}}, y, z)$. *Deep recursion* stacks T blocks with gradient flowing only through the last block (truncated BPTT), and *deep supervision* applies the loss at each of N_{sup} outer steps with a detached carry. We keep an EMA of the weights and a Q/halt head, as in TRM. The model has $\sim$221K parameters; a one-pass network with *identical* parameter count is the ablation control.

Inductive, not transductive. Unlike TRM/HRM/PTRM we use *no* per-instance identity embedding—only token and positional embeddings. The model therefore learns general constraint structure and is evaluated on a held-out set of instances it never saw; reported accuracy is genuine generalisation.

Verifier-gated inference-time scaling. Following PTRM we add Gaussian noise (σ) to the latent and run K parallel rollouts. We then select the output with a *deterministic verifier*—the problem's own feasibility checker—rather than the learned Q-head. For constraint problems a sound verifier is free.

Verifier-gated hybrid. At deployment the proposer runs K verifier- gated rollouts; if none passes, we fall back to an exact CP-SAT solve [7]. Every returned solution is thus either verifier-passed or exact—*sound by construction*—so the proposer can only improve cost, never correctness.

4 Experimental Setup

Tasks. (a) 4×4 Sudoku (8 of 16 cells blanked) as a validated CSP backbone; (b) **workforce shift rostering**: 6 workers $\times$ 5 days, each day requiring exactly 2 day and 1 night shift, a no-night-then-day rest rule, and at most 4 shifts per worker—the "fill-the-gap / shift-swap repair" problem common in managed-services workforce management. Instances are generated and, in the fallback path, solved by CP-SAT. Supervised learning requires label-consistent data, so training instances are filtered to *unique* completions (verified by CP-SAT solution counting).

Metric and hardware. The primary metric is *valid-solve / feasibility rate*: the fraction of held-out instances whose assembled grid passes the deterministic verifier. All runs are CPU-only (Apple silicon, 4 threads); no GPU is used.

5 Results

5.1 Recursion with deep supervision is the driver, at equal parameters

Table 1: Recursion vs. one-pass at *identical* parameter count ($\sim$221K).

Task	One-pass (same params)	Recursive TRM	Gain
Sudoku 4×4 (8 blanks)	81.0%	99.5%	+18.5 pp
Rostering 6×5 (6 blanks)	58.5%	81.5%	+23.0 pp

Both models share architecture and parameter count; the only difference is whether the core is recursed with deep supervision (Table 1, Figure 1). Recursion adds 18–23 points, reproducing on our own runs the central TRM claim: the gain comes from iterative refinement, not from parameters.

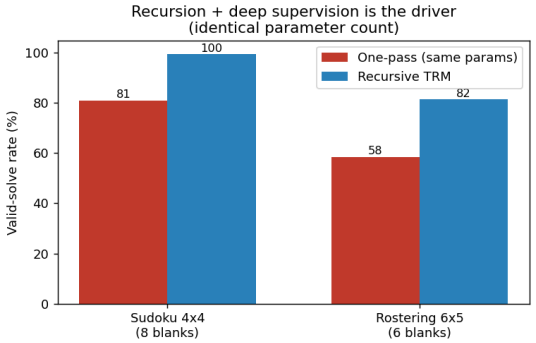


Figure 1: Recursion vs. one-pass at equal parameters.

5.2 Verifier-gated scaling, and the Q-head ceiling

Adding latent noise and running K rollouts, then selecting with the verifier, lifts the solve rate with no retraining (Table 2). Two effects match the literature. Verifier-gated selection climbs toward the pass@K oracle (Sudoku reaches 100%, rostering 91.5%). Crucially, selection by the *learned Q-head plateaus well below the verifier*—around 82% on rostering versus 91.5% for the verifier. The operational lesson: do not trust the internal ranking head; supply a deterministic verifier, which constraint problems natively provide.

5.3 The deployable pattern: verifier-gated proposer + exact-solver fallback

On 200 held-out rostering instances with K=16 rollouts (Table 3), the hybrid is sound by construction, achieves **100% feasibility**, avoids the exact solver on **89%** of instances, and is **5.0$\times$ faster** end-to-end than calling CP-SAT every time.

Table 2: Rostering 6×5 inference-time scaling (%). Verifier approaches the pass@K oracle; the Q-head does not.

K	1	4	16	32	64
pass@K (oracle)	81.0	87.5	90.0	91.0	91.5
verifier@K	81.0	87.5	90.0	91.0	91.5
Q-head best@K	81.0	81.5	82.0	79.5	79.5
majority@K	81.0	82.5	82.0	82.0	82.0

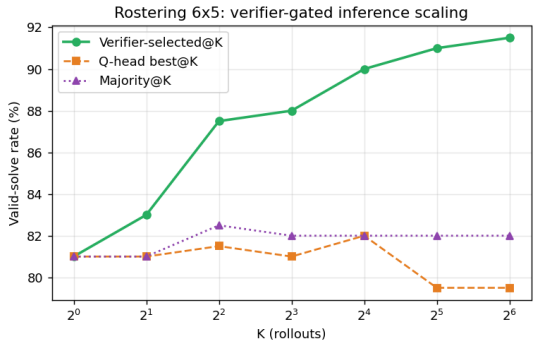


Figure 2: Rostering inference-time scaling: verifier tracks the oracle; the Q-head plateaus.

5.4 Scoped pilot: rigor, generalization, economics, and an SLM baseline

The results above are single-seed on a well-trained model. A multi-seed pilot (a leaner, strictly time-budgeted configuration) hardens them along four axes.

Statistical rigor. Across 5 seeds on 6×5, the proposer's single-shot valid rate is $48 \pm 8\%$ and verifier@16 is $59 \pm 7\%$: a spread wide enough that any single-seed number must be read as indicative. *Hybrid feasibility was 100% on every seed* (Figure 3a).

Generalization under shift. Training on 6 blanks and testing harder, proposer verifier@16 falls $60\% \rightarrow 42\% \rightarrow 34\%$ at 6/8/10 blanks, yet the hybrid stays **100% feasible** throughout—graceful degradation of efficiency, never of correctness.

Economics at scale (honest negative). Per attempt the proposer is ∼7–8× cheaper than a CP-SAT solve at every size, but the *realised* hybrid speedup falls with size ($1.92\times \rightarrow 1.13\times \rightarrow 0.88\times$ at 30/48/70 cells) because, under a fixed cheap training budget, proposer accuracy collapses ($48\% \rightarrow 28\% \rightarrow 2\%$) faster than CP-SAT slows—so more instances hit the fallback (Figure 3b).

Off-the-shelf SLM baseline. On the same 6×5 test set, a general 0.5B small language model [8] prompted few-shot (no fine-tuning) produced **0% feasible** rosters (52% parse rate) at 3.3 s/instance—over 2,000× more parameters than the recursive proposer, ∼30,000× slower per attempt, and unable to do the task (Figure 3c). A fine-tuned SLM would be a fairer, stronger baseline and remains future work.

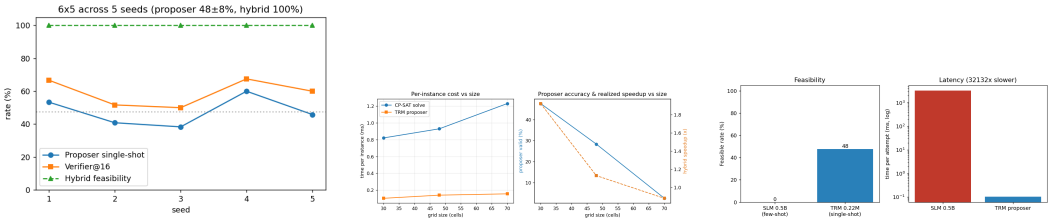
Verdict. Against the pre-registered rule (adopt only if the hybrid beats CP-SAT on cost at matched 100% feasibility across seeds and sizes): feasibility and robustness are *met*; cost/latency at scale is *not yet met*. The mechanism is sound and safe, and beats an off-the-shelf SLM decisively, but the economic case at scale is unproven under a cheap budget.

6 Discussion: Industrial Applicability

The result is about a *reusable pattern*, not rostering specifically. Any problem that is (i) a fixed-size set of decisions, (ii) governed by cheaply checkable hard rules, (iii) available in many similar

Table 3: Verifier-gated hybrid on 200 rostering instances (K=16).

Configuration	Feasibility	Solver calls	Total time (200 inst.)
CP-SAT only (exact)	100%	200 / 200	131 ms
TRM proposer only	89.0%	0	10 ms
Verifier-gated hybrid	**100.0%**	22 / 200	26 ms



(a) Multi-seed (6×5). (b) Cost & speedup vs. size. (c) SLM vs. TRM.

Figure 3: Pilot. (a) Hybrid is 100% feasible on all seeds despite proposer variance. (b) Per-attempt the TRM is far cheaper, but realised speedup falls with size under a cheap budget. (c) A 0.5B SLM is 0% feasible and orders of magnitude slower.

instances, and (iv) under latency, cost, privacy or edge pressure fits the tiny-proposer + verifier + exact-solver-fallback architecture. In **airline operations** this maps onto crew rostering and pairing, gate/stand assignment, tail assignment, and disruption recovery (where seconds-scale re-solves are the product). In **finance** it fits rule-checking and allocation—trade/order allocation under position limits, settlement netting and collateral assignment, pre-trade compliance validation, and desk scheduling. A $\sim$0.2M-parameter model runs on a laptop or edge device with no cloud dependency, aligning with bank data-residency and offline airport-operations constraints. Problems needing world knowledge or continuous optimisation (price prediction, credit decisioning, forecasting, natural-language handling) are *not* a fit.

7 Limitations

Our demonstrations are on small instances (up to 10×7); the economic case at scale is explicitly unproven (Section 5.4). Supervised per-cell training requires label-consistent (unique-completion) data; on the general many-completion distribution the proposer collapsed to $\sim$2%, so the verifier/solver—not the network—guarantees correctness in deployment. The SLM comparison uses an off-the-shelf model without fine-tuning. The next phase targets harder, solver-expensive instances, more training per size, multiple seeds throughout, and a fine-tuned SLM comparator.

8 Conclusion

A tiny, inductive recursive reasoning model, gated by a deterministic verifier and backed by an exact solver, is a credible, CPU-deployable pattern for industrial constraint satisfaction: it preserves 100% feasibility, removes most expensive solver calls, degrades safely under distribution shift, and decisively outperforms an off-the-shelf small language model. The honest caveat is that its realised economic advantage at scale is not yet established and is the explicit target of future work.

References

[1] A. Jolicoeur-Martineau. Less is More: Recursive Reasoning with Tiny Networks. *arXiv:2510.04871*, 2025.

[2] G. Wang et al. Hierarchical Reasoning Model. *arXiv:2506.21734*, 2025.

[3] Probabilistic Tiny Recursive Model (PTRM). *arXiv:2605.19943*, 2026.

[4] Deep-neural-network-guided tree search for personnel rostering. *arXiv:2010.14252*, 2020.

[5] Machine-learning-based repair of nurse schedules. *Journal of Advanced Mechanical Design, Systems, and Manufacturing*, 2026.

[6] I. Tolstikhin et al. MLP-Mixer: An all-MLP Architecture for Vision. *NeurIPS*, 2021.

[7] L. Perron and V. Furnon. OR-Tools CP-SAT Solver. Google, 2024. https://developers.google.com/optimization.

[8] Qwen Team. Qwen2.5 Technical Report. *arXiv:2412.15115*, 2024.